Information about information series

Marianne Freiberger

Table of Contents

Author: Marianne Freiberger

Date: 24 March, 2015

1 Information is surprise

It's not very often that a single paper opens up a whole new science. But that's what happened in 1948 when Claude Shannon published his *Mathematical theory of communication*. Its title may seem strange at first — human communication is everything but mathematical. But Shannon wasn't thinking about people talking to each other. Instead, he was interested in

the various ways of transmitting information long-distance, including telegraphy, telephones, radio and TV. It's that kind of communication his theory was built around.

Shannon wasn't the first to think about information. Harry Nyquist and Ralph Hartley had already made inroads into the area in the 1920s (see this article), but their ideas needed refining. That's what Shannon set out to do, and his contribution was so great, he has become known as the father of information theory.

Shannon wanted to measure the amount of information you could transmit via various media. There are many ways of sending messages: you could produce smoke signals, use Morse code, the telephone, or (in today's world) send an email. To treat them all on equal terms, Shannon decided to forget about exactly how each of these methods transmits a message and simply thought of them as ways of producing strings of symbols. How do you measure the information contained in such a string?

It's a tricky question. If your string of symbols constitutes a passage of English text, then you could just count the number of words it contains. But this is silly: it would give the sentence "The Sun will rise tomorrow" the same information value as the sentence "The world will end tomorrow" when the second is clearly much more significant than the first. Whether or not we find a message informative depends on whether it's news to us and what this news means to us.

Shannon stayed clear of the slippery concept of meaning, declaring it "irrelevant to the engineering problem", but he did take on board the idea that information is related to what's new: it's related to surprise. Thought of in emotional terms surprise is hard to measure, but you can get to grips with it by imagining yourself watching words come out of a ticker tape, like they used to have in news agencies. Some words, like "the" or "a" are pretty unsurprising; in fact they are redundant since you could probably understand the message without them. The real essence of the message lies in words that aren't as common, such as "alien" or "invasion".

This suggests a measure of surprise for each word: the *frequency* with which it appears in the English language. For example, the most frequent word in the 100 million word British National Corpus is the word "the", appearing, on average, 61,847 times in a million words. The word "invasion" appears, on average, only 19 times per million words, while "alien" is not even listed, presumably because it is so rare.

You could simply measure the amount of surprise you feel when a word occurs by the number of times it occurs per million words in some large volume of English text. But this would be missing a trick. The word "invasion" is pretty surprising when it follows the word "alien",

but less surprising when it follows the word "military". Perhaps surprise should be measured, not only in terms of the overall frequency of a word, but also in terms of the word that came before it (you can do this by looking at the frequency of pairs of words in large body of text). To capture even more of the structure within the English language, you could base your surprise measure on the frequency of triples, quadruples, quintuples, and so on.

1.1 Producing gibberish

Shannon appears to have had some fun playing around with this idea. On page 7 of *A mathematical theory of communication* he reproduced a string of words that were picked at random, independently of each other and with a probability reflecting their frequency:

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TOOF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE.

He also produced a sentence in which the probability of a word occurring depended on its predecessor, based on the frequencies of pairs of words:

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED.

Both sentences are utterly meaningless, but the second looks just that little bit more like an English sentence than the first: there are whole strings of words that actually make sense.

But Shannon's games with random words had another purpose too. If you picked a more sophisticated probability distribution for your words, one that picks up on more structure within the English language, you could presumably approximate that language much more closely. Conversely, you could think of any device that produces strings of words, like the ticker machine, as a *stochastic* process. Forget about the person who is sending the message and the meaning they want to convey and simply imagine the machine picking words at random, based on a probability distribution that closely reflects the structure of the English language. To formalise things even further, forget about words and think of the machine picking individual symbols according to a certain probability distribution. This way, we can measure surprise not only for languages spoken by people, but also for strings of 0s and 1s produced by a computer, or for smoke signals sent in the forest.

1.2 Calculating surprise

This line of thinking led Shannon to consider an idealised situation. Suppose there's a random process which produces strings of symbols according to a certain probability distribution, which we know. Assume, for the moment, that each symbol is picked independently of the one before. We could simply define the surprise $s(x)$ associated to a single symbol x as the reciprocal of its probability $p(x)$, so $s(x) = 1/p(x)$, reflecting the fact that the less probable a letter, the more surprised we are at seeing it. But this definition leads to a problem. The probability of seeing two symbols x and y occurring one after the other, if they are picked independently, is the product of the individual probabilities, $p(x) \times p(y)$. If we measure surprise by reciprocals of probabilities, then the surprise associated to the two symbols appearing together is the product of the individual surprises

$$s(xy) = \frac{1}{p(xy)} = \frac{1}{p(x) \times p(y)} = \frac{1}{p(x)} \times \frac{1}{p(y)} = s(x) \times s(y).$$

However, intuitively the surprise of seeing x followed by y should be the sum of the individual surprises. You can convince yourself of this by imagining that I tell you two facts that aren't related to each other, for example "the cat is black" and "today is Monday". Because they are unrelated the total amount of surprise contained in both sentences is the information of the first plus the information of the second. A function that does make surprises *additive* is the *logarithm* of the reciprocal of probabilities:

$$s(x) = \log\left(1/p(x)\right).$$

We still have the inverse relationship (the less probable a symbol the greater the surprise) and the surprise of seeing x and y appear one after the other is now

$$s(xy) = \log\left(1/(p(x)p(y))\right) = \log\left(1/p(x)\right) + \log\left(1/p(y)\right) = s(x) + s(y).$$

According to this definition the surprise associated to seeing a longer string is simply the sum of the individual surprises. The logarithm of inverse probability also has some other properties that sit well with our intuition of surprise. For example, if the machine always produces the same letter x, so $p(x) = 1$, then we shouldn't be surprised at all to see x, and indeed in this case the surprise is 0:

$$s(x) = \log 1 = 0.$$

Thus, Shannon decided to stick to the logarithm as a measure of surprise. It doesn't really

matter which base you choose for the logarithm — the choice varies the exact surprise value for each symbol, but doesn't change the way they interact. (Note that if there are n symbols, all with equal probability $1/n$, then the surprise of each is exactly Hartley's measure of information introduced in the earlier precursor article.)

1.3 Average surprise

This sorts out the amount of surprise, which we've related to the amount of information, of a string of symbols produced by our machine. For reasons that will become clear later, we can also calculate the *expected* amount of surprise the machine produces per symbol. This is akin to an average, but takes into account that symbols with higher probabilities occur more often, and should therefore contribute more to the average than those with low probabilities (see here to find out more about expected values).

If our machine produces n letters, x_1, x_2, x_3, etc up to x_n, with probabilities p_1, p_2, p_3, etc up to p_n, then the expected value of surprise is

$$H = p_1 \log(1/p_1) + p_2 \log(1/p_2) + p_3 \log(1/p_3) + ... + p_n \log(1/p_n).$$

By the rules for the logarithm, we know that $\log(1/p_i) = -\log(p_i)$ so we can write H as

$$H = -(p_1 \log p_1 + p_2 \log p_2 + p_3 \log p_3 + ... + p_n \log p_n).$$

If you know some physics then this expression might look familiar. It looks exactly the same as a quantity called *entropy* which physicists had defined around seventy years earlier to understand the behaviour of liquids and gases (find out more in this article). This parallel wasn't lost on Shannon. He called the measure of average information H defined above the *entropy* of the machine. It depends only on the probability distribution of the possible symbols, the exact workings of the mechanism producing it don't matter. Entropy is a truly universal measure of information.

(If you've been paying attention, you might have noticed one issue. To define our measure of surprise we assumed that symbols were being picked independently of each other. Earlier, however, we said that to simulate a real message written, say, in English, the probability of a symbol being picked should depend on the symbol or, even better, symbols that came before it. But this isn't a problem. There's a way of picking those dependent probabilities apart to get many independent distributions — the overall entropy is then an average of the component ones. See Shannon's paper to find out more.)

1.4 Entropy of a coin flip

Let's look at an example. Suppose the machine can only produce two symbols, h for heads and t for tails, and that it picks them with equal probability of $1/2$. It might do so by flipping a fair coin, hence the choice of the symbols. The entropy of this probability distribution is

$$H = 1/2 \log(2) + 1/2 \log(2) = \log(2).$$

Choosing the base of the logarithm to be 2, this gives us a nice round value:

$$H = \log_2(2) = 1.$$

Now suppose that the coin is crooked, so it comes up heads 90% of the time. This means that $p(h) = 0.9$ and $p(t) = 0.1$. The entropy now becomes

$$H = 0.9 \log_2(1/0.9) + 0.1 \log_2(1/0.1) = 0.469.$$

It's lower than the entropy of the fair coin and this makes sense: knowing that the coin is crooked, we can guess the outcome correctly around 90% of time by betting on heads. Therefore, on average, our surprise at seeing the actual outcome isn't that great — it's definitely less than for a fair coin. Thinking of surprise as measuring the information, this tells us that the average amount of information inherent in the machine printing one symbol is less than the corresponding number for the fair coin.

But why on Earth would you want to calculate the average amount of surprise (or information) per symbol? It turns out that Shannon's entropy also has a very concrete interpretation that is very relevant to the modern world: it measures the minimum number of bits — that's 0s and 1s — needed to encode a message. The next section explores that idea in more detail.

2 Information is bits

We send information over the internet every day. All that information, whether it's a birthday email or your homework, is in encoded into sequences of 0s and 1s. What's the best way of doing this? The way that uses the least amount of 0s and 1s and therefore requires the smallest amount of computer memory.

Let's do a back-of-the-envelope calculation. Suppose we want to encode the 26 (lower case) letters of the alphabet, plus an additional symbol that denotes a space: each of the 27 symbols is to be represented by a string of 0s and 1s. How long do these strings need to be? Clearly

they must be long enough to give a total of at least 27 distinct strings, so that we can associate them uniquely to our 27 symbols. There are 2^m strings of length m (because we have two choices for each entry in the string) so we need

$$2^m \geq 27.$$

Solving for m gives

$$m \geq \log_2 27 \approx 4.75.$$

This suggests that we need 5 bits per character to encode each of our 27 symbols. You can make a similar calculation for the 45,000 characters of the Mandarin language, giving

$$\log_2 45,000 \approx 15.46$$

and suggesting that you need 16 bits per character to encode them. We still don't know the exact code to use, but at least we know how heavy it's going to be on our computer memory.

2.1 Exploiting patterns

The calculation above is neat, but we can do better. "Any reasonable [code] would take advantage of the fact that some letters, like the letter"e" in English, occur much more frequently than others," explains Scott Aaronson, a computer scientist at the Massachusetts Institute of Technology. "So we can use a smaller number of bits for those." It's an idea that's been used in Morse code for over 150 years: here the more common letters are encoded using shorter strings of dots and dashes than the rarer ones.

As an example, suppose there are only three letters to encode, A, B and C. Suppose that A has frequency 1/2 (in a long text half of the letters will be A) and B and C have frequency 1/4 each (in a long text a quarter of the letters will be B and another quarter will be C). Since A is the most frequent, let's use a single bit, say 0, to encode it. For B and C we will use two bits each, say 10 and 11. You can check for yourself that this is a decent code — the receiver can decode it to find the original letter sequence without any ambiguity. To encode a text made up of those letters, we need one bit per symbol half of the time (for the A), two bits per symbol a quarter of the time (for the B) and two bits per symbol another quarter of the time (for the C). So the average number of bits per symbols is

$$1/2 \times 1 + 1/4 \times 2 + 1/4 \times 2 = 3/2 = 1.5.$$

We can even guess at a more general formula that works for different frequency distributions.

To see how, let's first stick to our example of A, B and C, and imagine that half of the time the letter A occurs it is red and that the other half it is green. This gives four possibilities: a red letter A with frequency 1/4, a green letter A with frequency 1/4, an ordinary black letter B with frequency 1/4, and a black letter C with frequency 1/4. All the frequencies are now the same, so if we wanted to encode the characters in a way that distinguishes the colour of the A's we would need around

$$\log_2 4$$

bits per character. That's the same calculation as the one above. In particular, we need $\log_2 4$ bits to identify the B and the C. For the letter A, the number $\log_2 4$ is overkill: we don't actually need to distinguish between the two colours. So let's deduct the number of bits needed to distinguish between two different characters (corresponding to the two different colours), which is $\log_2 2$. This gives

$$\log_2 4 - \log_2 2 = \log_2 \frac{4}{2} = \log_2 2$$

bits to represent the A. The average number of bits per symbol is now

$$\frac{1}{2}\log_2 2 + \frac{1}{4}\log_2 4 + \frac{1}{4}\log_2 4 = 1/2 + 1/2 + 1/2 = 3/2 = 1.5$$

That's the same number we got before, which is reassuring. But we also notice that each term in the sum above is the frequency of one of the characters (e.g. 1/2 for A) times the logarithm of 1 divided by the frequency (e.g. $\log_2 2$ for A). And it turns out that exactly the same way of thinking works in general for an alphabet of n symbols. Writing p_1 for the frequency of symbol 1, p_2 for the frequency of symbol 2, and so on, an estimate for the average number of bits needed per symbol is

$$p_1 \log(1/p_1) + p_2 \log(1/p_2) + p_3 \log(1/p_3) + ... + p_n \log(1/p_n).$$

2.2 The birth of the bit

Thinking of information in terms of bits — 0s and 1s — seems like something that firmly belongs to the modern era, but the idea has been around for over 60 years. It was the mathematician Claude Shannon who in 1948 popularised the word "bit", which is shorthand for "binary digit". Shannon also came up with the formula we found above. Given an alphabet of n symbols and a probability distribution telling you the probability p_i with which a symbol i occurs in a text made out of those symbols, the number

$$H = p_1 \log(1/p_1) + p_2 \log(1/p_2) + p_3 \log(1/p_3) + ... + p_n \log(1/p_n)$$

is called the *entropy* of the distribution, as discussed in the previous section. Shannon proved that the average number of bits needed per symbol cannot be smaller than the entropy, no matter how cleverly you encode them. For the vast majority of alphabets and distributions, the average number of bits needed is in fact only slightly bigger than the entropy. (One encoding that comes very close to Shannon's entropy for the vast majority of alphabets and distributions is the so-called Huffman code.)

If you're in the business of sending messages long-distance, the entropy is a useful number to know. If you know you can transmit some number C bits per second, and that the symbols in your message require around H bits per symbol on average, then you'd guess that you can transmit around C/H symbols per second on average. Shannon showed that this is indeed correct: by picking a clever way of encoding your symbols you can guarantee an average transmission rate that's as close to C/H per second as you like. Conversely, no matter how you encode your symbols, it's impossible to transmit at a faster rate than C/H. This neat fact is known as Shannon's *source coding theorem*.

Shannon published this result in *A mathematical theory of communication*, "A paper that is justly celebrated as one of the greatest of the twentieth century," says Aaronson. It's a highly theoretical effort. The source coding theorem, for example, doesn't tell you which code gets you within a specified distance of the maximal rate of C/H per second — it only says that there is one.

When it comes to practical applications, the theorem also has a major short-coming: it assumes that your communication channels transmit pristine messages and never introduce any errors. In real life this just doesn't happen: whether you send a message in a telegram, via email or using SMS, there is always a chance it is scrambled on the way and arrives at the other end corrupted. This is where things get interesting. The next section turns to so-called *noisy channels* and asks how much information you can transmit through them.

3 Information is noisy

Our channels of communication aren't perfect. Whether you send a message by email or by smoke signal, there's always the chance that some random interference mangles it up on the way. Surprisingly though, you can always reduce the error rate to nearly zero, simply by choosing a clever way of encoding the message.

3.1 Flipping bits

To get a feel for this, imagine you want to transmit the outcome of a coin flip to me via some long-distance communication channel, writing a 0 for heads and a 1 for tails. You know there's a chance that a 0 will get changed into a 1 on the way, and vice versa. Can you come up with a code that is secure against such *bit flipping*? You might try using two bits per outcome, writing 00 for heads and 11 for tails. If a 01 or a 10 arrives at the other end, then I will know that an error has happened. But I won't know which of the two bits was flipped, so I can't deduce whether you originally sent a 00 or a 11.

Let's instead try a code using three bits; 000 for heads and 111 for tails. If I see a sequence that isn't one of those two I won't only know that there has been an error. Assuming that only one bit per three can be flipped, I will also know what triple was the original: if the erroneous string has two 0s then the original was 000, otherwise it was 111. Such a way of encoding, which secures against errors, is called an *error correcting code*.

The example illustrates an important idea. You can get to grips with errors by introducing *redundancy*: extra symbols that reduce the chance that an error corrupts the message. "This idea shows up all over the place," says Scott Aaronson, a computer scientist at the Massachusetts Institute of Technology. "The English language, or any other language, has lots of redundancy built into it. If you miss one little piece of what I said you can still understand the sentence. This is a property of human languages: we don't maximally compress the information for a good reason."

3.2 Errors versus redundancy

Just how error-proof can you make things using such error-correcting codes? In our example above the error rate would be zero if we could be sure that the channel only ever flips one bit per block of three. In reality things aren't that predictable: you might know that your channel flips one bit per three *on average*, but this doesn't mean that it can't occasionally change a 000 standing for heads into a 011, leaving the receiver none the wiser as to what the intended message was. You might look for a cleverer code to guard against this, but the point is that whatever code you use, there's still a chance of ambiguity, leading to an error in the decoded message at the other end.

It seems that to reduce that chance, all you can do is add more redundancy. This would also make your transmission much less efficient since you need more bits to transmit a single symbol. It also puts you in danger of entering an arms race. If I ask you to produce a code that reduces the error rate to some small number, you need to add a certain amount of redundancy, if I reduce my required error rate further, you need to add more redundancy,

an even smaller error rate requires even more redundancy, and so on, beyond all limits. A near-zero error rate would then require a near-infinite amount of redundancy, which means near-zero efficiency.

In the earlier days of long-distance communication this was indeed what people thought: when you're dealing with a noisy channel, you have no choice but trade your error rate for redundancy. In 1948, however, the mathematician Claude Shannon proved them wrong. In his ground-breaking *Mathematical theory of communication* Shannon showed that given any error rate, no matter how small, it's possible to find a code that produces this tiny error rate and enables you to transmit messages at a transmission rate that is only limited by the channel's capacity. This result is known as Shannon's *noisy channel coding theorem*. It shows that near-error-free transmission doesn't lead to near-zero efficiency.

As the previous section suggests, the noisy channel theorem involves the entropy of the source that produces the information (such as a coin flipping machine). Loosely speaking, the entropy measures the average amount of information carried by a symbol produced by the machine. Adding redundancy to a message reduces the amount of information per symbol and therefore lowers the entropy. What Shannon showed is that once the entropy is below some critical value, a code with an arbitrarily small rate of decoding error can always be found. That critical value is the channel's *capacity* — the maximum rate at which the channel can convey information given its noise characteristics.

3.3 Finding the best code

How do you find that magic code? Unfortunately, Shannon doesn't tell us. "Shannon didn't give a single concrete example of what code you would use," says Aaronson. His proof merely shows that if you pick a code at random from a set of suitable candidate codes, then with overwhelming probability that code will do. It's quite an amazing result, but not a very useful one. "Picking a code at random would not be computationally efficient. You might need a prohibitive amount of computational power to decode it." But Shannon wasn't concerned with that; his aim was to see what could be done with noisy channels in principle.

Which brings us right up to date. "A lot of the research in information theory in the 60-plus years since Shannon's paper, you can think of as an attempt to make Shannon's result explicit: trying to come up with an explicit error correcting code that is able to transmit information at close to the [maximal] rate that Shannon's theoretical construction gives you," says Aaronson. "There has been a lot of progress, especially in the last twenty-five years. There are now codes that can be encoded and decoded very efficiently and come close to [Shannon's limit]. These codes have major applications. They are used in CDs, in sending

messages to space probes, etc. They are used all over the place in electrical engineering."

Shannon's ideas have been hugely influential, but his way of measuring information has a curious feature. The entropy measures the average information of a symbol produced by a particular information source. It tells us nothing about the information contained in one string of symbols produced by the source compared to another. Surely there must be a way of measuring that too? The next section takes up that question.

4 Information is complexity

How much information is there in this section? It's hard to tell. Its length clearly isn't an indicator because that depends on my choice of words. One thing you could do to measure information is to condense the argument into bullet points. If you end up with many of them, then that's because the text is pretty dense. If there's only one, then the information it conveys is rather basic, or at least not very complex.

4.1 Measuring complexity

In the 1960s mathematicians developed a way of measuring the complexity of a piece of information that reflects this intuition. They were looking at information from a computer's point of view. To a machine, a piece of information is just a string of symbols: letters, numbers, or, at the most basic level, 0s and 1s. A computer can't make bullet points, but it can be programmed to output a given string of symbols. If the information in the string can be condensed quite a lot, then there should be a short program that does this. If it can't, then the program will be longer.

To see how this works, imagine the string consists of a hundred zeros. A program that outputs that string only needs to say something like "print 0; repeat 100 times". Different programming languages will use different commands to do this, but the general idea will be the same. So that's a short program for a pretty simple string. If, on the other hand, your string contains 0s and 1s without any pattern at all, then there is no structure to exploit. Your program would have to say "print:" and then quote the entire string, making the program slightly longer than the string itself.

The length of the shortest computer program that produces a given string and then halts (we don't want infinite loops) is called the *Kolmogorov complexity* of the string, after Andrey Kolmogorov, one of three people who came up with the idea (the other two were Ray Solomonoff and Gregory Chaitin). If you interpret complexity as lack of structure, then the name makes sense.

"Kolmogorov complexity measures the maximum compressibility of a string via any computable process whatsoever," says Scott Aaronson, a computer scientist at the Massachusetts Institute of Technology. "Compressibility is something we're all familiar with. There are all sorts of data compression programs, which play a crucial role on the internet, for example for streaming videos." The programs exploit the fact that lots of the information we deal with has some sort of structure, like the string of 100 zeroes above. The English language obviously contains structure, and so do images and pieces of music (which to a computer are also strings of 0s and 1s). Just as with the string of 0s above, you can exploit this structure to compress a file.

4.2 Computing complexity...or not

So, what's the Kolmogorov complexity of your favourite song or your favourite movie? Before working it out, you need to fix which programming language you use, since a program could be shorter in one language than in another. This is a bit like fixing units of measurement such as inches or centimetres. It's important to know which units you're using, but once you've agreed on them, you can stop referring to them all the time. Which is why we won't specify any particular language here. The only restriction is that you're not allowed to cheat, for example by saving your favourite string under the name "s" and then writing a program that simply says "print s" — we require that the program contains all the instructions on how to compress and decompress information.

Next, you need to find the shortest program that outputs your song or movie, and that's a problem. "Kolmogorov completely left open the question of how you actually find the shortest program to output a given string," says Aaronson. That's bad news, but things get worse. "You can actually prove that this is an *uncomputable problem*." Informally, this means that there is no general method for computing the Kolmogorov complexity of any given string. You might be able to compute it for some strings, but there's no general recipe that works for all.

There's a very elegant proof of this fact, based on a logical conundrum called *Berry's paradox*. First notice that I could use ordinary English words to specify a number. For example, the sentence "The number of fingers on a generic person's hand" specifies the number five. Clearly not every number can be specified using a short sentence, say of at most 20 words. I, for one, can't think of a brief way to specify the number 2,354,899,123,657. Amongst the numbers that can't be specified by a sentence of at most 20 words, there must be a smallest one, which I'll call n.

But hang on — I've just specified n using the sentence

Sentence A: "n is the smallest whole number that cannot be specified using at most 20 words",

which itself has less than 20 words. Therefore n can and can't be specified using at most 20 words — it's a paradox. Bertrand Russell was the first to write about this puzzle, but he had learnt it from a junior librarian in Oxford by the name of G. G. Berry.

The way out of the conundrum lies in the ambiguity of language. Sentence A doesn't give you any numerical value, it merely gives a name, n, to a number you believe exists. If giving a name to a number you believe exist counts as specifying it, then you can specify any number at all, so there isn't a smallest one you can't specify. If you require that specifying a number involves something more concrete, then sentence A doesn't actually specify n. In either case, the paradox unravels.

There is no such ambiguity in computer languages though, which is why an analogue of Berry's paradox proves that Kolmogorov complexity is uncomputable. "Let's suppose there were a [general method for] computing the Kolmogorov complexity of any given string," explains Aaronson. "Then it turns out that you could use this method to design a very small computer program that finds a string of enormous Kolmogorov complexity, much bigger than the length of the program itself." That's a contradiction. The Kolmogorov complexity of the output string is the length of the shortest program that produces it. Since our program does produce the string, the string's Kolmogorov complexity can't possibly exceed the program's length: it's got to be the same, or smaller. We've encountered a contradiction, and this time there's no way out by appealing to ambiguity. We have no choice but to conclude that our original assumption, that Kolmogorov complexity is computable, was false.

4.3 Uncomputable but useful

What's the use of a measure you can't actually compute? "Kolmogorov complexity is useful because it tells you the ideal situation you would like to approach with the imperfect tools you have in practice," says Aaronson. "In situations where the maths calls for Kolmogorov complexity, you can instead feed your data into a compression program, say using gzip or saving your image as a .gif file, and use that as a crude approximation of Kolmogorov complexity. Kolmogorov complexity would be the theoretical ideal compression. What you're getting with standard compression methods are various non-ideal compressions, which use various regularities in your data, but not all regularities."

The theoretical concept has inspired some very practical applications. Imagine, for example, you want to compare two DNA strings, or two music or image files, to see if they are correlated in some way. One thing to do is to take the strings x and y representing the two files and to stick one to the end of the other, creating one very long string xy. If x and y are exactly the

same, so they are maximally correlated, then the Kolmogorov complexity of xy will be very close to that of the individual Kolmogorov complexity of x and y. That's because a computer program that outputs xy would only need to include the program that outputs x and print the output twice.

If, on the other hand, x and y aren't correlated at all, then the shortest computer program to produce xy would have to consist of, roughly, the shortest program that outputs x and the shortest program that outputs y. So in that case the Kolmogorov complexity of xy approximately equals the Kolmogorov complexity of x plus the Kolmogorov complexity of y.

In practice you can't compute the Kolmogorov complexity, but you can use compression programs to approximate it. If the approximated complexity of xy is close to the sum of the approximated individual complexities of x and y, then that's evidence they are not very correlated. If it's close to the complexity of an individual string, it's evidence that they are.

"There are all sorts of reasons why you might want to know the general relatedness of two different products, and this is often a useful technique," says Aaronson. "These are not direct applications because you cannot compute Kolmogorov complexity. But the maths gives you the ideal, which then gives you inspiration for what you can do with the tools you have."

But how does Kolmogorov complexity hold up as a measure of information content? The complexity of a string is low if it contains a lot of regularities that can be exploited for compression. If it doesn't contain regularities, in other words if it is random, then the Kolmogorov complexity is high. But can random strings really be thought of as carrying a lot of information? Aren't they in some sense meaningless? The final section looks at that question through the idea of sophistication.

5 Information is sophistication

It's almost miraculous to think that the most evocative song or exciting movie can be captured in a sober string of 0s and 1s, to be stored and processed by computers. Zeroes and ones are the backbone of information technology — but where within those strings does the information they represent really lie?

It's a question people have been asking themselves for a surprisingly long time. Back in the 1940s the mathematician Claude Shannon realised that information is only informative when it's telling you something unexpected, and decided to measure the information in strings of symbols in terms of how predictable they are, as discussed above. In the 1960s mathematicians drew on the fact that many words don't necessarily mean a lot of content, and decided to measure information in terms of how much you can compress it. The *Kolmogorov*

complexity of a string of 0s and 1s is the length of the shortest computer program that can produce it.

Complexity might indeed be a good measure of information: the more complex something is, the more information it contains. But Kolmogorov complexity has a curious feature. Suppose your string consists of a thousand zeroes. A computer program that outputs that string only needs to say "print 0; 1000 times", so it's pretty short, leading to a low Kolmogorov complexity. This makes sense, as the string is very basic.

Now imagine a completely random string of a thousand zeroes and ones; one that doesn't exhibit any pattern at all. A program that outputs that string has no choice but to say "print:" and then quote the entire string. There is no pattern here to exploit to make the program any shorter. Long random strings have high (in fact maximal) Kolmogorov complexity. (Shannon's measure similarly assigns maximal entropy to a *source* whose output is completely unpredictable.) But does this make sense? Patternless strings may be hard to describe, but don't we usually associate complexity with some sort of structure, albeit a complex one?

"Kolmogorov complexity is misnamed because really it just measures patternlessness," says Scott Aaronson, a computer scientist at the Massachusetts Institute of Technology. "If we are actually trying to measures complexity, we need something that is small for very simple strings and also small for completely patternless strings. Then there should be some third class of strings for which that measure is not small."

As an example, look at the string `000011111100110011`.

Does it contain a pattern? After staring at it for a while, you'll probably spot that it's made out of double bits 00 and 11 - you never see a 0 or a 1 occur alone. The order in which the 00s and 11s occur, however, seems random. Perhaps a measure of complexity and/or information should evaluate the amount of structure within this string, the doubling of bits, ignoring the random part. That way, completely random strings would have a low complexity value, as they have no structure at all, and so would strings with a very simple structure. Strings that contain more complex patterns would have a higher value.

This is exactly the idea behind a concept called *sophistication*, which emerged in the 1980s. The idea is to look at programs that can produce a given string, taking some other information as input. In the example above, the input could be the string 001110101, and the program could be "double each bit". It's a short program, reflecting the fact that the structure in this case (each bit appears double) is simple. A more complex structure (say a random string made up of the triples 010 and 101) would lead to a longer program.

Loosely speaking, the *sophistication* of a string s is the length of the shortest program that outputs s given some random string as input, and then halts (no matter the value of the input). The input reflects the random, and in a sense uninteresting, part of the string, while the program reflects its structure. How do we decide whether the input string is "random"? We can use Kolmogorov complexity to do this: we require that the Kolmogorov complexity of the input string be not much smaller than its length (in technical language, such a string is called *algorithmically random*).

5.1 Is anything sophisticated?

This seems neat, but it poses an awkward question. As we've seen, there are strings with low sophistication: those with simple structure and completely random ones. But are there any strings to which our new measure assigns a high value? Are there strings with a lot of sophistication? Strangely, mathematicians have only been able to find few such strings, and these are highly contrived, arising from esoteric logical arguments. They are not random, but because they involve intricate logical problems, their structure is incredibly hard to describe.

"This is unsatisfactory," says Aaronson. "If every string you ever come across in real life [as opposed to abstract maths] is unsophisticated, then maybe we're setting our standards too high. The trouble is, if a string is going to arise in real life, then you can ask 'how did it arise'? Presumably it arose by some combination of natural laws and the injection of some random component. If the law part is simple, and we like to think of the laws of physics as simple, and the random part is not being counted, then how are you ever going to get [high] sophistication?"

As an example, imagine recording the time the Sun reaches a particular point in the sky each day. The resulting string of numbers may exhibit a little randomness due to small measuring errors, but it's going to have a lot of structure, given by the comparatively simple equations that govern planetary motion. The string (once it's encoded in binary) will have low sophistication because a program could compute it using those simple equations. "The very fact that a string was generated by a process we can understand, logically implies that the string is unsophisticated," says Aaronson.

This brings us right up to date with current research. Computer scientists are busy refining notions of complexity and sophistication to make them more amenable to real-life measurements. One approach is not to ask for *the* shortest computer program that outputs a string given some input, but for the shortest computer program that does so in some reasonable amount of time. "A huge number of things are abstractly computable, but it would take longer than the age of the Universe to compute them," explains Aaronson. Such monstrous

programs are useless in practical terms, so we might want to exclude them when computing sophistication. Although they take ages to run, they might actually be very short in length, so excluding them could only ever increase the sophistication of a given string. This may create those elusive high-sophistication strings.

This idea has already been fairly well developed for Kolmogorov complexity. "I think the right answer to our question is to study sophistication [in a similar way]. Unfortunately that theory really has not been well developed yet," says Aaronson.

When engineers first started thinking about information in the 1920s, the idea of treating it using maths, rather than psychology, was completely new. Ralph Hartley's pioneering 1928 paper, *Transmission of information* has an entire section entitled *Elimination of psychological factors*. Today, we're much more comfortable with thinking of information in such sober terms. But with so much research still going on, it seems that its real secret still hasn't been cracked.